a

	Which clones it can use			How well it describes clones						Computer resources					Count
	Share of clones it describes	Keeps two-cell clones	Datasets it could analyse	Predicts measured clone traits	… when traits come from other cells	Recognises a clone from other cells	Links day-4 clones to day 6	Finds planted clone groups	Tells mixed from in-between clones	Run time	Peak memory	Gain from 16 CPU cores	Time for 10× the clones	Runs that ended without help	Tiles better / worse
Threadfin (donor as context, as in the case studies)	100 %	100 %	11 of 11	reference	reference	11 %	0.030	0.72	0.87	3.1 s (median)	0.5 GB (median)	2.0× (analysis)	20× time (profiles)	77 of 77	—
clone2vec, its default (clones ≥ 3 cells)	▼ 50 %	▼ 0 %	11 of 11	10 better 3 worse	11 better 7 worse	31 % vs 55 %	0.110 vs 0.087	▲ 0.92 vs 0.85	▲ 0.98 vs 0.87	▼ 27× slower	▼ 2.3× more	▼ 1.2× vs 2.0×	▲ 6× vs 20×	65 of 65	3 / 5
clone2vec, forced to clones ≥ 2 cells	100 %	100 %	11 of 11	5 better 5 worse	17 better 12 worse	▼ 3.6 % vs 11 %	▲ 0.040 vs 0.030	▲ 0.74 vs 0.72	▲ 0.98 vs 0.87	▼ 34× slower	▼ 2.3× more	▼ 1.2× vs 2.0×	▲ 6× vs 20×	77 of 77	4 / 4
Clonotrace, its default (clones ≥ 10 cells)	▼ 8.5 %	▼ 0 %	▼ 7 of 11 4 too small	▼ 0 better 6 worse	5 better 13 worse	39 % vs 49 %	0.098 vs 0.087	0.83 vs 0.87	0.62 vs 0.88	▼ 2,436× slower	▼ ≥ 3.8× more	▲ 9.0× vs 2.0×	▼ 164× vs 20×	▼ 19 of 21 2 re-run	1 / 8
Clonotrace, forced to clones ≥ 2 cells	▼ 64 %	▼ 32 %	11 of 11	2 better 7 worse	10 better 11 worse	8.9 % vs 21 %	▲ 0.039 vs 0.030	0.52 vs 0.62	—	▼ 286× slower	▼ ≥ 7.4× more	▲ 9.0× vs 2.0×	▼ 164× vs 20×	▼ 26 of 29 3 re-run	2 / 6
Clonotrace, exact OT (≥ 2 cells; ≥ 10 where only that ran)	▼ 60 %	▼ 32 %	▼ 6 of 9 3 failed	1 better 3 worse	not run on halves	not run on halves	cancelled	0.52 vs 0.62	not run	▼ 204× slower	▼ 13× more	—	—	▼ 8 of 12 4 failed	0 / 6
Benisse	100 %	100 %	9 of 9	▼ 3 better 23 worse	6 better 15 worse	8.3 % vs 14 %	no BCR	no BCR	no BCR	▼ 33× slower	1.6× more	▼ 0.34× vs 2.0×	▼ 123× vs 20×	▼ 26 of 27 1 failed	0 / 5
CoNGA's one-cell-per-clone rule	100 %	100 %	11 of 11	▼ 1 better 15 worse	9 better 20 worse	3.3 % vs 11 %	▼ 0.019 vs 0.030	▼ 0.44 vs 0.72	▼ 0.49 vs 0.87	not timed	not measured	—	—	77 of 77	0 / 4
Mean of the embedding (baseline)	100 %	100 %	11 of 11	7 better 5 worse	16 better 13 worse	8.2 % vs 11 %	0.034 vs 0.030	▼ 0.67 vs 0.72	▼ 0.57 vs 0.87	not timed	not measured	—	—	77 of 77	0 / 2
Cluster composition (baseline)	100 %	100 %	11 of 11	▼ 1 better 20 worse	▼ 4 better 25 worse	3.4 % vs 11 %	▼ 0.015 vs 0.030	▲ 0.75 vs 0.72	▲ 0.93 vs 0.87	not timed	not measured	—	—	77 of 77	2 / 3
Threadfin with another random seed (negative control)	—	—	—	2 better 2 worse	▲ 21 better 8 worse	8.8 % vs 9.3 %	—	0.72 vs 0.72	▼ 0.78 vs 0.87	—	—	—	—	—	1 / 1

▲ better than Threadfin | no clear difference | ▼ worse than Threadfin | Threadfin's own value | too few datasets or seeds to test

Each tile compares the tool with Threadfin on the same clones; the numbers are the tool's and, after "vs", Threadfin's. Paired tests ($P < 0.05$) over annotations, datasets or simulation seeds; single measurements count as different beyond 25 %. —: no such output. Rules: README, section 0. The last row shows how often the rules call a difference between two runs of Threadfin that differ only in their random seed.

b

	Get it				Trust the code			Learn it		Use it safely			Credibility	
	Open-source licence	Numbered releases	One-command install	Depend-encies versioned	Automated tests	Tests on every change	Same result, any cores	Step-by-step tutorial	Docu-mented functions	Explains bad input	Uses several cores	AnnData or Seurat	Peer-reviewed	Score
Threadfin	■	■	■	■	■	■	■	■	■	■	■	■	□	12 / 13
Clonotrace	■	▒	▒	■	■	■	■	■	■	▒	■	▒	□	10 / 13
clone2vec	■	■	■	■	■	■	▒	■	■	▒	■	■	□	11 / 13
Benisse	■	■	□	▒	□	□	▒	▒	▒	□	▒	□	■	5.5 / 13
CoNGA	■	▒	□	▒	□	□	not assessed	■	▒	not assessed	not assessed	■	■	5.5 / 10
scirpy	■	■	■	■	■	■	not assessed	■	■	not assessed	■	■	■	11 / 11

■ yes ▒ partly □ no

The authors' assessment; evidence for every score in scripts/usability_checklist.csv. Threadfin is not yet peer-reviewed.